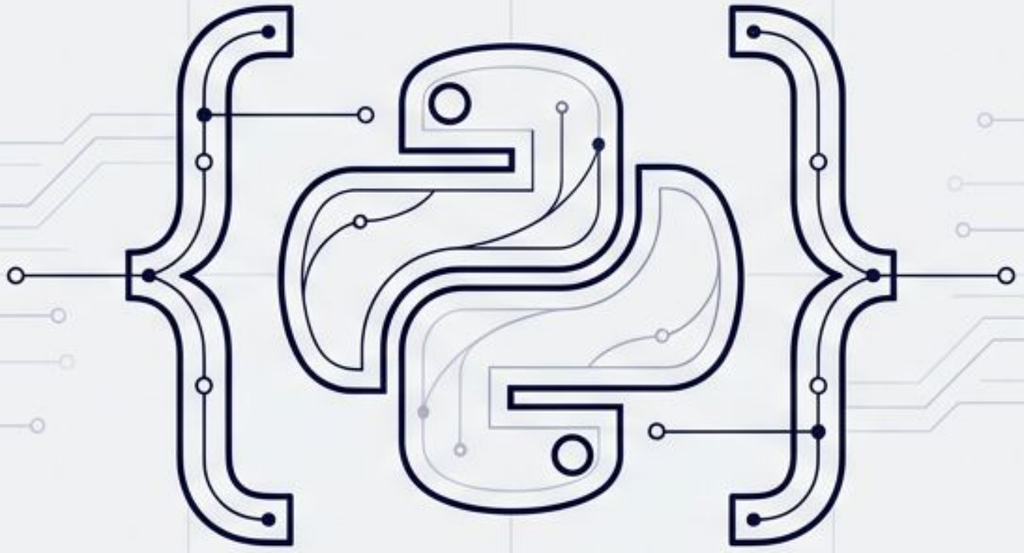
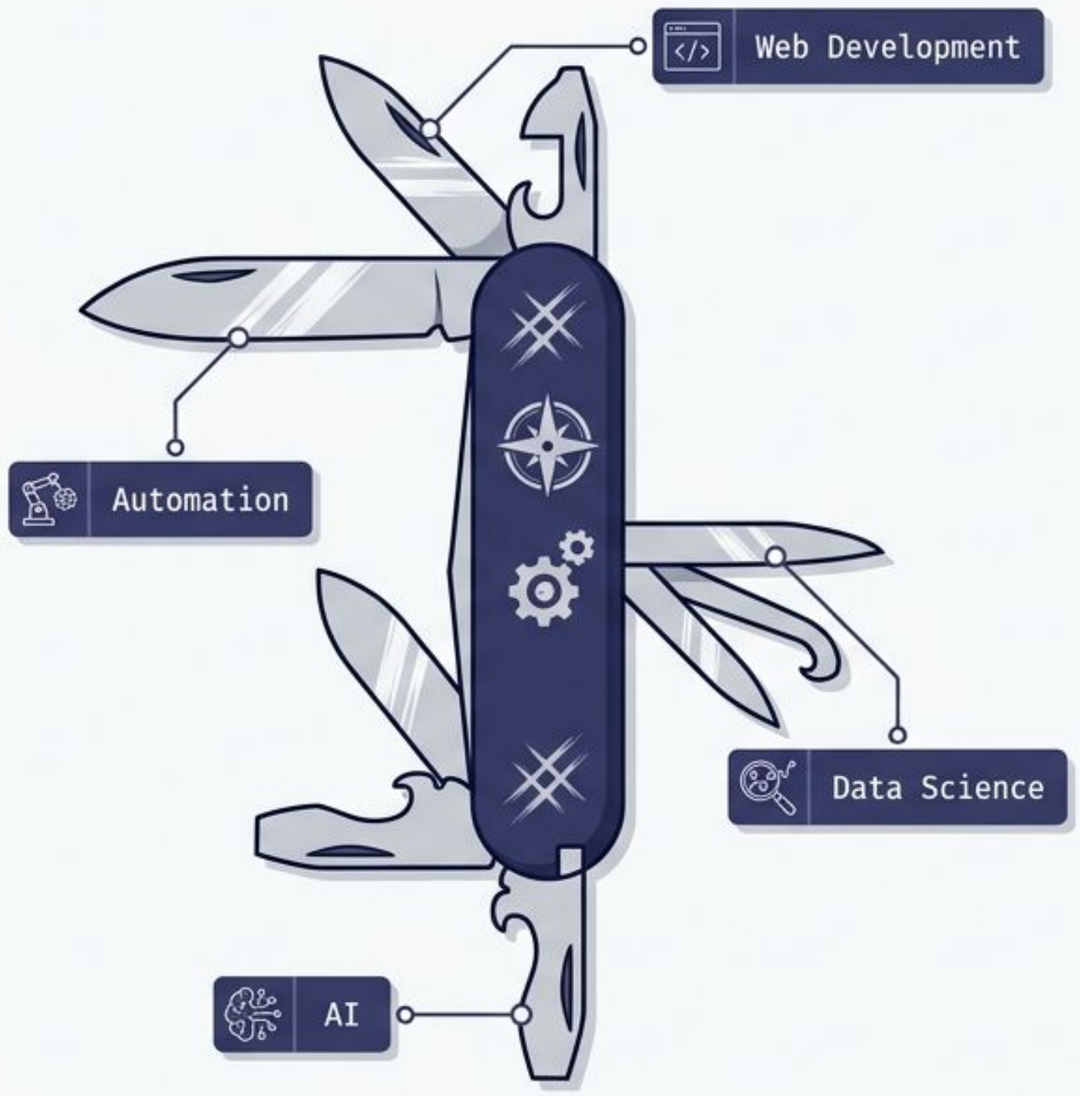


Fundamentals of Python Programming

The Complete Exam-Preparation Blueprint (SBTE Bihar Diploma)

```
Loading module: Unit_1... [OK]
```



The Blueprint Flow

The Foundation
(Architecture & History)
Understanding the core design principles, historical context, and fundamental architecture of the system. Includes overview of technical evolution.

1

The Environment
(Setup & Execution)
Configuring the development ecosystem, toolchain installation, and mastering the execution lifecycle. Emphasizes reproducible environments.

2

The Building Blocks
(Variables & Identifiers)
Deep dive into naming conventions, variable scope, and memory allocation for different identifiers. Focus on clean, readable code.

3

The Materials
(Data Types & Memory)
Comprehensive study of primitive and complex data structures, memory management, and type systems. Includes stack vs. heap analysis.

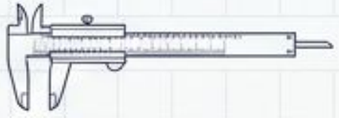
4

The Wiring
(Operators & Logic)
Mastering control flow, logical operators, and decision-making constructs. Includes best practices for writing efficient, conditional code.

5

Synthesis
(Ultimate Revision Sheet)
Consolidating all concepts into a unified, high-yield review resource for rapid recall. The final integration step.

6



READY >>

The Origin Story

Creator: Guido van Rossum.
Known as the BDFL (Benevolent Dictator for Life) until 2018.

1989: Started at CWI, Netherlands

1991: v0.9 Released

1994: v1.0

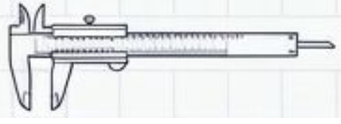
2000: v2.0

2008: v3.0 (Major redesign)

2020: v2 officially discontinued

The Name?
Named after the British comedy 'Monty Python's Flying Circus', not a snake.

Quick Revision: 1989 (started) -> 1991 (v0.9) -> 2008 (v3.0). Creator = Guido van Rossum (BDFL).



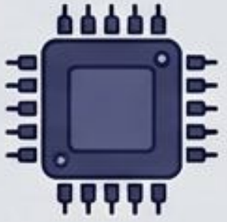
Why Python is ‘High-Level’

The Human

The Assistant:
Python Interpreter
(CPython)

Abstracts hardware & manages memory automatically.

The Hardware:
Machine Code (0s and 1s)



Executes the task.

```
name = 'Rahul'
print('Hello,', name)
```

Viva Question: High-level means closer to human language. It hides hardware complexity and requires an interpreter.

Core Features of Python

Simple Syntax: English-like, minimal punctuation.

Open Source: Free to use and modify.

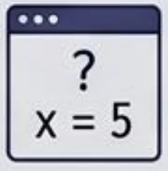
Interpreted: Executes line-by-line; no separate compilation.

Portable: Same code runs on Windows, Linux, Mac.

Object-Oriented: Supports classes and inheritance.

High-Level: Hardware abstraction.

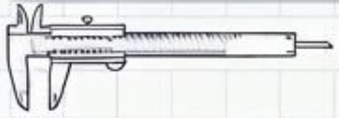
Extensive Libraries: Massive built-in standard library.



Dynamically Typed: No explicit variable declarations needed.

Extensible: Integrates with C/C++/Java.

Exam Point: Examiner requires definition + minimum 3 reasons with examples for full marks on ‘High-Level’ and ‘Features’ questions.



The Execution Pipeline & Python Path

Download from python.org

↓

Run Installer (.exe)

↓

⚠ **CRITICAL STEP:**
Check "Add Python to PATH"

↓

Verify via Command Prompt:
python --version

What is the PATH?

PATH is an OS environment variable.

Adding Python to it allows the interpreter to launch directly from any terminal folder.

Exam Point: Default IDE is IDLE, but PyCharm and VS Code are popular alternatives.

The Anatomy of a Python File

.py

The Source Code
Human-readable script.
Identifies file to the OS and IDE (enables syntax highlighting).

.pyc

Compiled Bytecode
What the interpreter generates behind the scenes.

.pyw

Windows GUI Script
Runs without opening a console window.

Analogy: Just as .docx tells the OS to open Word, .py tells the OS to launch the Python Interpreter.

Quick Revision: Write .py with one-line reasoning for 1-mark PYQ. Used for source code and module importing.

First Execution: "Hello, World!"

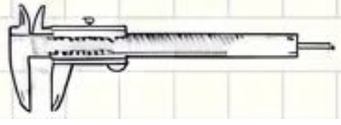
hello.py

```
1  # A high-level, simple syntax example
2  print('Hello, World!')
```

Indicates a comment

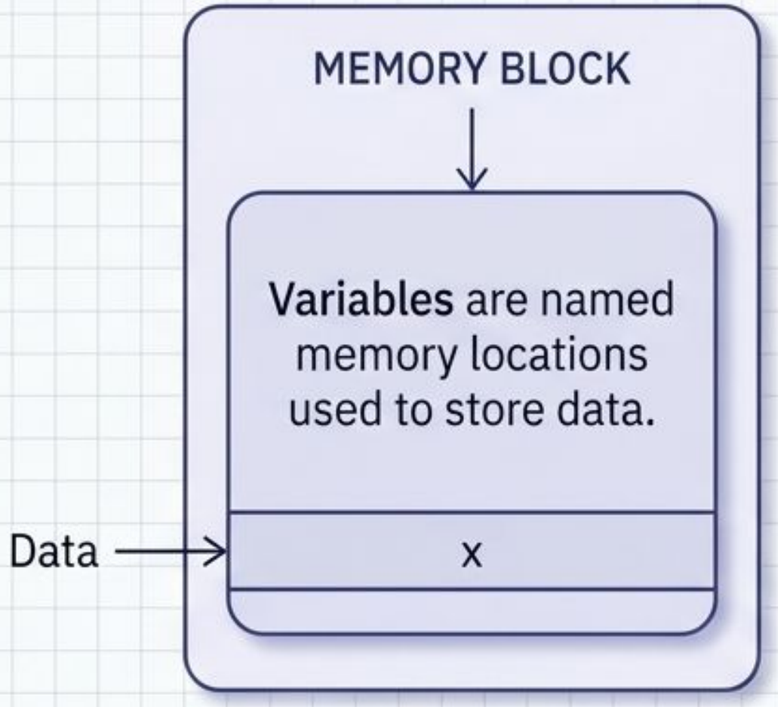
Built-in output function

```
> python hello.py
Hello, World!
```



Variables & Dynamic Typing

The Concept



The ‘Dynamic’ Feature

Python is **dynamically typed**. The type is decided automatically at runtime based on the assigned value.

Terminal

```
>>> x = 10        # Created as int
>>> type(x)
<class 'int'>
>>> x = 'Hello'   # Automatically changes to string
>>> type(x)
<class 'str'>
>>>
```

Assignment 1: Integer

Assignment 2: String

Quick Revision: Variables are created automatically upon assignment. No need to declare “int x;” like in C.

Variable Naming Rules: The Traffic Light

Valid

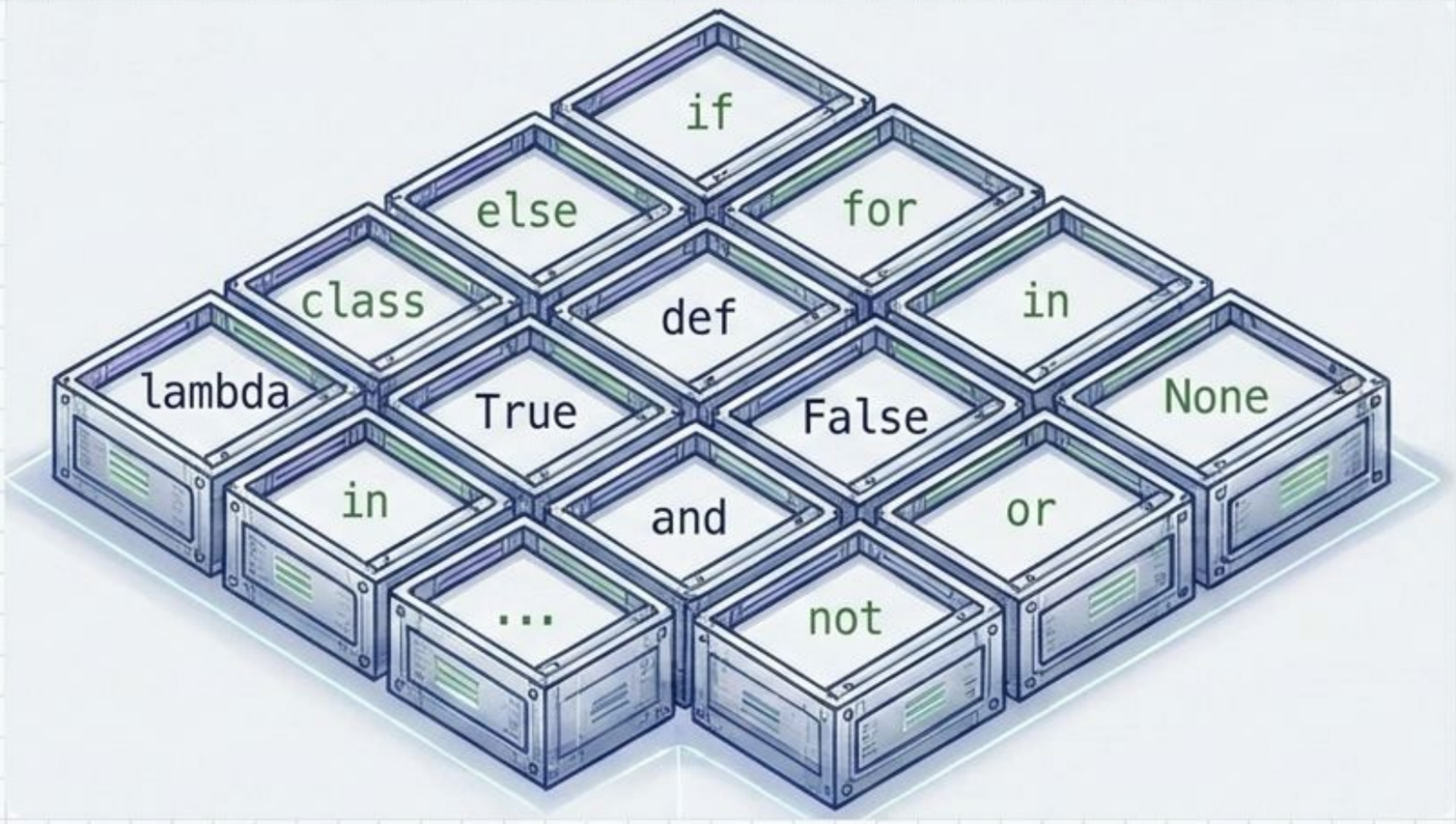
✓ `name1 = 'Ram'`	(Letters and numbers fine)
✓ `_first_name = 'Sita'`	(Underscore is the only valid symbol)
✓ `firstName = 'Geeta'`	(Case-sensitive)

Invalid

✗ `1name = 'Ram'`	(Cannot start with a digit)
✗ `first name = 10`	(Spaces not allowed)
✗ `class = 'A'`	(Cannot be a reserved keyword)
✗ `first-name = 'Sita'`	(Hyphens not allowed)

Exam Point: Rule: Letter/underscore start -> only letters/digits/underscore -> not a keyword -> case sensitive.

Keywords: The Restricted Vault



The Concept

Keywords are reserved words with predefined meanings.

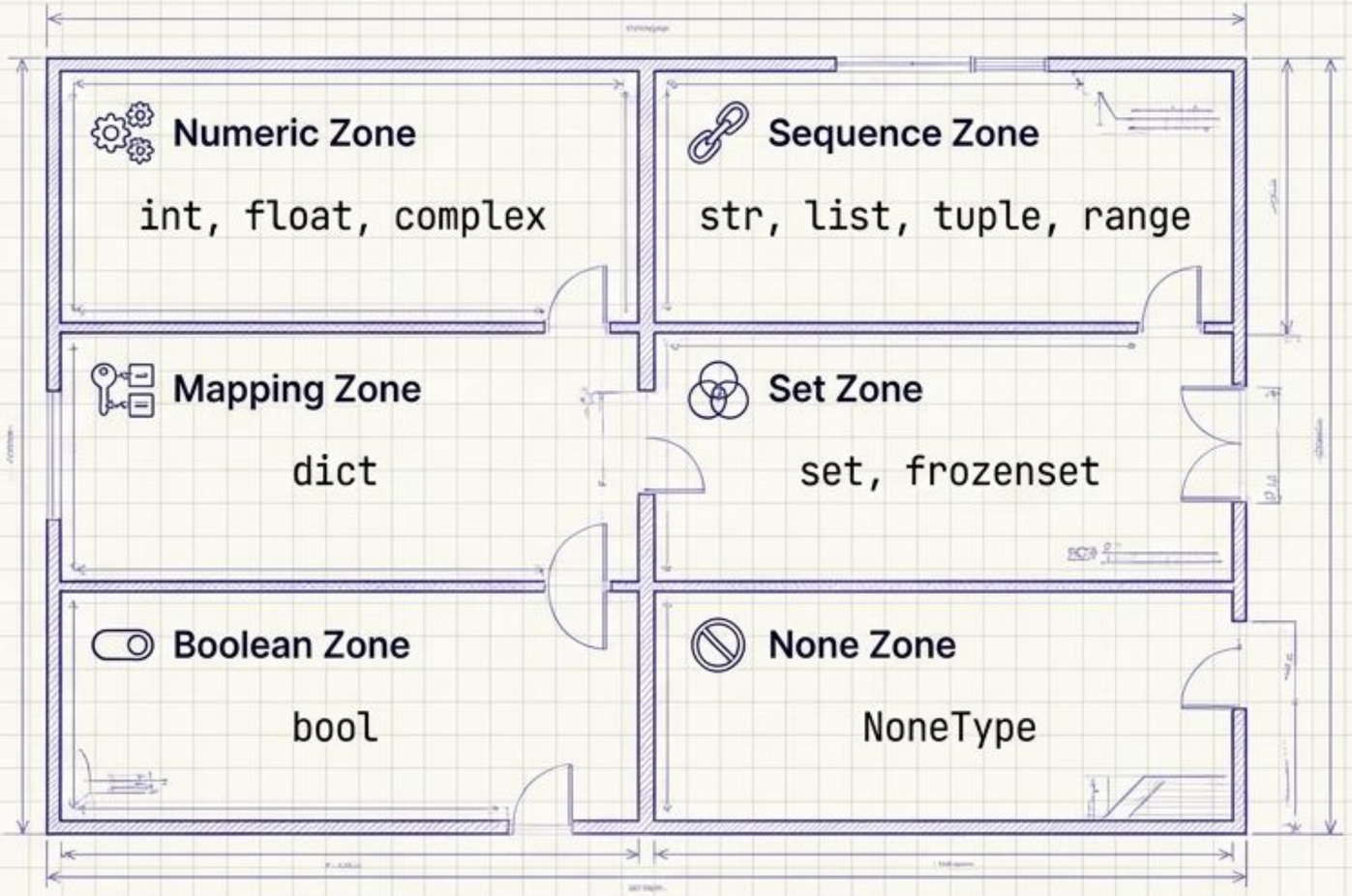
They are locked by the system and cannot be used as variable names or identifiers.

Python has exactly **35 standard keywords**.

Viva Question: What is an identifier? A name used to identify a variable, function, or class, distinct from restricted keywords.

The Data Type Warehouse

Like a warehouse has distinct sections for electronics, groceries, and clothes, Python has distinct types for different data structures.



Exam Point: High-weightage subjective question (4-6 marks). **Always draw this classification table for full marks.**

Code Canvas: Data Types in Action

```
a = 10           # int
c = 3 + 4j       # complex
e = [1, 2, 3]    # list
f = (1, 2, 3)    # tuple
g = {'key': 'v'}# dict
i = True         # bool
print(type(c), type(e), type(f))
```

```
<class 'complex'>
<class 'list'>
<class 'tuple'>
```

Quick Revision: type() is a built-in function that reveals the dynamic classification of any variable at runtime.

The Memory Architecture: Mutable vs. Immutable

Mutable = The Whiteboard. Content can be erased and changed without changing the physical board (memory address).

`list`, `dict`, `set`

Immutable = The Printed Page. To change the text, you cannot edit the paper; you must print an entirely new page (new memory object).

`int`, `float`, `str`, `tuple`, `bool`

Proving Mutability via id()

Mutable List

```
my_list = [1, 2, 3]
my_list.append(4)
```

```
1402...1200 -> 1402...1200 (ID remains the same)
```

Immutable String

```
my_str = 'hello'
my_str = my_str + ' world'
```

```
1402...2345 -> 1402...3456 (ID changes!)
```

Exam Point: Always mention the id() function and give one program-based proof when asked about mutability.

Type Conversion: Implicit vs. Explicit

Implicit Conversion

Automatic conversion by Python to avoid data loss.

```
10 (int) + 5.5 (float)
= 15.5 (float)
```

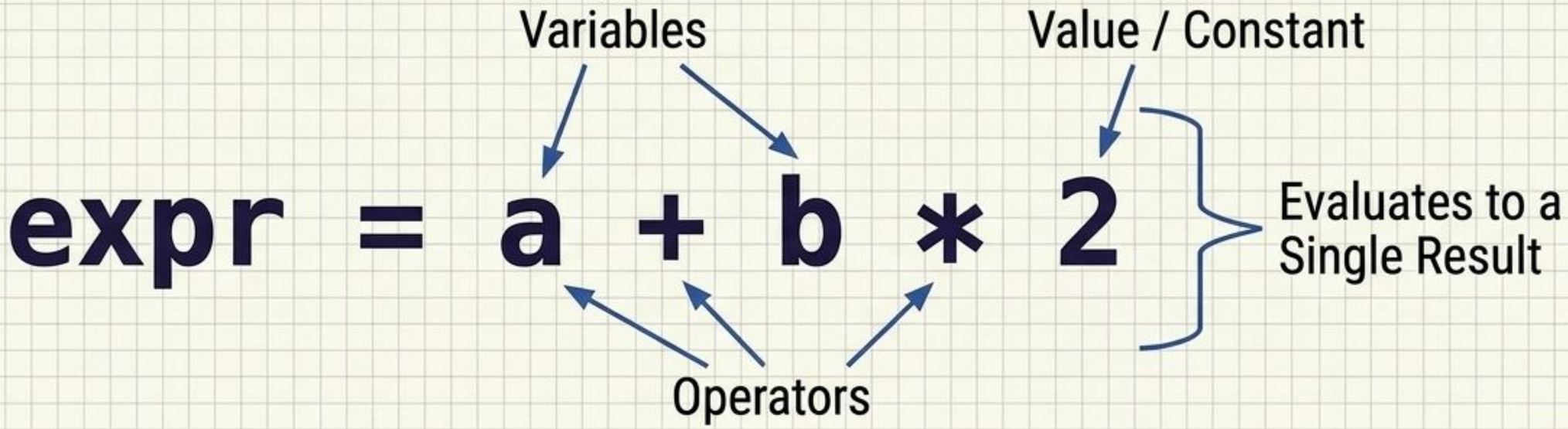
Explicit Conversion / Type Type Casting

Manual override using functions like `int()`, `float()`, `str()`.

```
int('25') + 10 = 35
```

Quick Revision: Implicit = Safe, automatic background conversion.
Explicit = Forced manual conversion.

Anatomy of an Expression



Just like a formula "Area = length * width", an expression is any legal combination of values, variables, and operators that evaluates to produce a single value.

Viva Question: How does an expression differ from a statement?
An expression evaluates to a value; a statement performs an action.

The Operator Matrix

Arithmetic

`+ - * / // % **`

Relational

`== != > < >= <=`

Logical

`and or not`

Assignment

`= += -= *= /=`

Identity

`is, is not`

Membership

`in, not in`

Quick Revision: Relational operators compare values and return a Boolean (True/False).

Arithmetic Deep Dive (PYQ Targets)

Floor Division //

```
17 // 3
```

Discards the decimal.

17 ÷ 3 = 5.66

→ Result: **5.**

Modulus %

```
17 % 3
```

Gives the remainder.

3 × 5 = 15,

Remainder → Result: **2.**

Exponent **

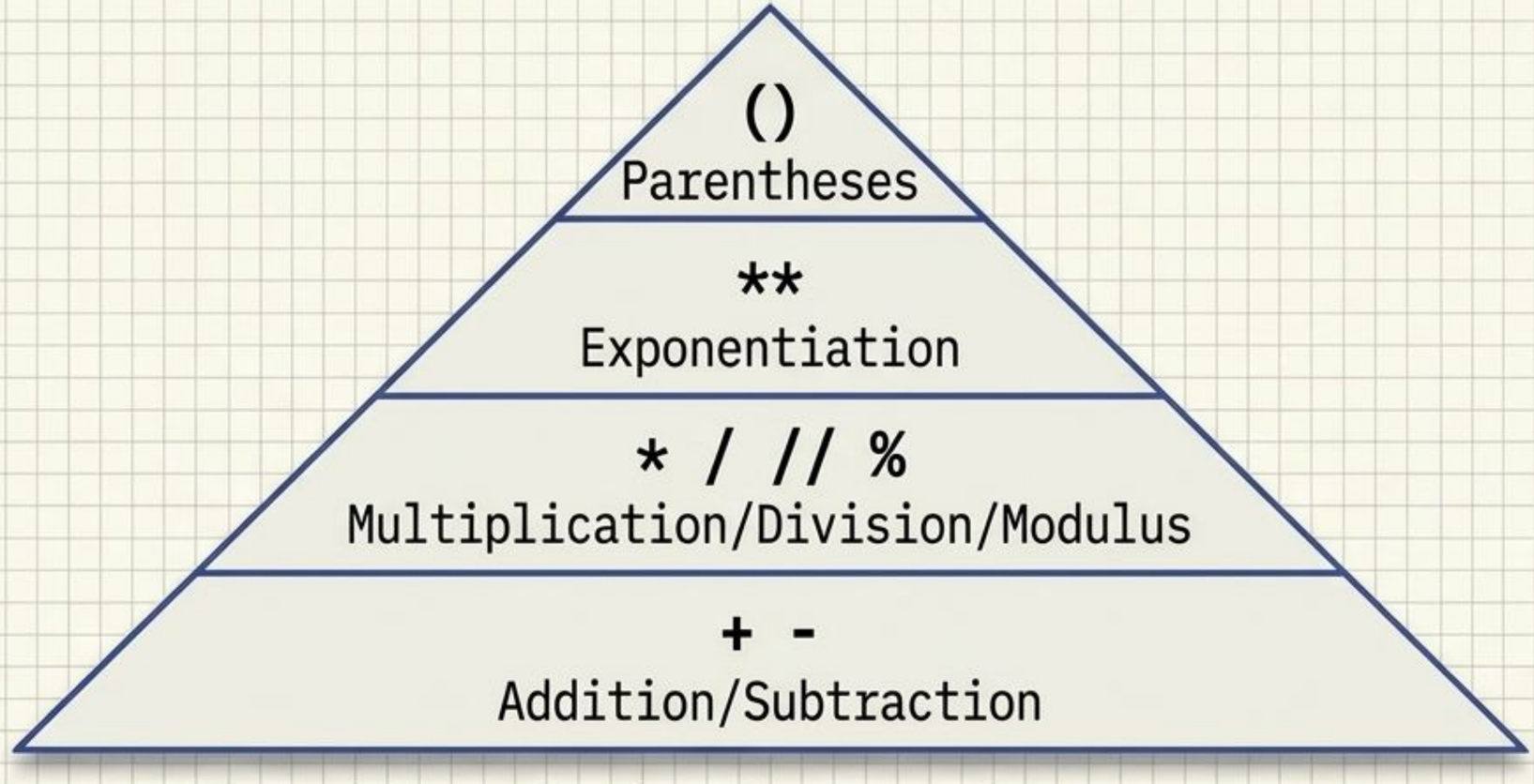
```
2 ** 3
```

Power calculation.

2 × 2 × 2 → Result: **8.**

Exam Point: These specific math operations (17//3, 17%3, 2**3) have appeared repeatedly in SBTE objective MCQs.

Operator Precedence Hierarchy



Followed by Relational -> Logical operators.

Memory Trick: Master floor division, modulus, and exponent. They are evaluated before basic addition.

Membership Operators (in, not in)

Checks whether a value is found in a sequence.

```
fruits = ['apple', 'banana']
print('banana' in fruits) # True
print('grapes' not in fruits) # True

text = 'Python Programming'
print('Prog' in text) # True
```

Exam Point: Always show both a string and a list example to get full subjective marks for this 2024 PYQ.